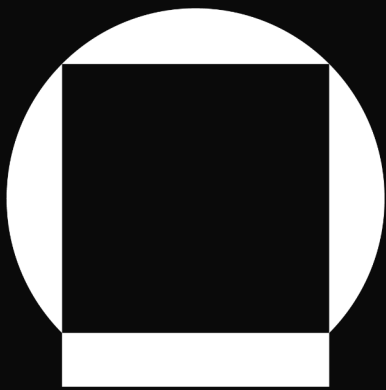




Security Assessment

Final Report



galaxy

Liquid Collective Protocol

March 2026

Prepared for Alluvial Finance

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Out of Scope:.....	6
Protocol Overview.....	6
Assessment Methodology.....	7
Threat and Security Overview.....	8
Assets.....	8
Trust Assumptions.....	10
Findings Summary.....	13
Severity Matrix.....	13
Detailed Findings.....	14
High Severity Issues.....	16
H-01: Denylisted users can bypass freeze controls by unwrapping wLsETH via burn.....	16
Medium Severity Issues.....	18
M-01: Missing On-Chain Validator Pubkey Uniqueness Allows Permanent Asset Balance Inflation.....	18
M-02: Inconsistent initiator deny check allows bypass via River redeem path.....	20
Low Severity Issues.....	21
L-01: Slashing containment mode still allows fresh validator funding.....	21
L-02: Phantom Pre-Exiting Balance From Unfulfilled Exit Requests.....	23
L-03: Firewall intercepts Administrable functions, locking destination contract ownership until proxy upgrade.....	26
L-04: Direct LsETH transfers to the wrapper strand underlying and break wLsETH supply accounting.....	27
Informational Severity Issues.....	28
I-01: `removeValidators` silently modifies operator limit without emitting SetOperatorLimit event, creating divergence between on-chain state and off-chain monitoring.....	28
I-02: MAX_VALIDATOR_ATTRIBUTION_PER_ROUND limitation has been removed, without any documentation.....	29
Disclaimer.....	30
About Certora.....	30

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Final Commit Hash	Platform
Liquid Collective Protocol	liquid-collective/liquid-collective-protocol	dbcec79986	964f0e363f	Solidity

Project Overview

This document describes the security review of **Liquid Collective Protocol**. The work was undertaken from **February 26th, 2026** to **March 16th, 2026**.

The following contract list is included in our scope:

contracts/src/Administrable.sol
contracts/src/Allowlist.1.sol
contracts/src/CoverageFund.1.sol
contracts/src/ELFeeRecipient.1.sol
contracts/src/Firewall.sol
contracts/src/Initializable.sol
contracts/src/OperatorsRegistry.1.sol
contracts/src/Oracle.1.sol
contracts/src/ProtocolMetrics.1.sol
contracts/src/RedeemManager.1.sol
contracts/src/River.1.sol
contracts/src/TUPProxy.sol
contracts/src/Withdraw.1.sol
contracts/src/components/ConsensusLayerDepositManager.1.sol
contracts/src/components/ERC20VestableVotesUpgradeable.1.sol

contracts/src/components/OracleManager.1.sol
contracts/src/components/SharesManager.1.sol
contracts/src/components/UserDepositManager.1.sol
contracts/src/libraries/LibAdministrable.sol
contracts/src/libraries/LibAllowlistMasks.sol
contracts/src/libraries/LibBasisPoints.sol
contracts/src/libraries/LibBytes.sol
contracts/src/libraries/LibErrors.sol
contracts/src/libraries/LibSanitize.sol
contracts/src/libraries/LibUint256.sol
contracts/src/libraries/LibUnstructuredStorage.sol
contracts/src/state/allowlist/AllowlistAddress.sol
contracts/src/state/allowlist/Allowlist.sol
contracts/src/state/allowlist/DenierAddress.sol
contracts/src/state/operatorsRegistry/CurrentValidatorExitsDemand.sol
contracts/src/state/operatorsRegistry/Operators.1.sol
contracts/src/state/operatorsRegistry/Operators.2.sol
contracts/src/state/operatorsRegistry/TotalValidatorExitsRequested.sol
contracts/src/state/operatorsRegistry/ValidatorKeys.sol
contracts/src/state/oracle/LastEpochId.sol
contracts/src/state/oracle/OracleMembers.sol
contracts/src/state/oracle/Quorum.sol
contracts/src/state/oracle/ReportsPositions.sol
contracts/src/state/oracle/ReportsVariants.sol
contracts/src/state/redeemManager/BufferedExceedingEth.sol
contracts/src/state/redeemManager/RedeemDemand.sol
contracts/src/state/redeemManager/RedeemQueue.1.sol
contracts/src/state/redeemManager/RedeemQueue.2.sol
contracts/src/state/redeemManager/WithdrawalStack.sol
contracts/src/state/river/AllowlistAddress.sol
contracts/src/state/river/BalanceToDeposit.sol
contracts/src/state/river/BalanceToRedeem.sol
contracts/src/state/river/CLSpec.sol
contracts/src/state/river/CLValidatorCount.sol

contracts/src/state/river/CLValidatorTotalBalance.sol
contracts/src/state/river/CollectorAddress.sol
contracts/src/state/river/CommittedBalance.sol
contracts/src/state/river/CoverageFundAddress.sol
contracts/src/state/river/DailyCommittableLimits.sol
contracts/src/state/river/DepositContractAddress.sol
contracts/src/state/river/DepositedValidatorCount.sol
contracts/src/state/river/ELFeeRecipientAddress.sol
contracts/src/state/river/GlobalFee.sol
contracts/src/state/river/KeeperAddress.sol
contracts/src/state/river/LastConsensusLayerReport.sol
contracts/src/state/river/LastOracleRoundId.sol
contracts/src/state/river/MetadataURI.sol
contracts/src/state/river/OperatorsRegistryAddress.sol
contracts/src/state/river/OracleAddress.sol
contracts/src/state/river/RedeemManagerAddress.sol
contracts/src/state/river/ReportBounds.sol
contracts/src/state/river/Shares.sol
contracts/src/state/river/SharesPerOwner.sol
contracts/src/state/river/WithdrawalCredentials.sol
contracts/src/state/shared/AdministratorAddress.sol
contracts/src/state/shared/ApprovalsPerOwner.sol
contracts/src/state/shared/PendingAdministratorAddress.sol
contracts/src/state/shared/RiverAddress.sol
contracts/src/state/shared/Version.sol
contracts/src/state/slashingCoverage/BalanceForCoverage.sol
contracts/src/state/tlc/IgnoreGlobalUnlockSchedule.sol
contracts/src/state/tlc/VestingSchedules.1.sol
contracts/src/state/tlc/VestingSchedules.2.sol

Out of Scope:

contracts/src/I2-token
contracts/src/TLC.1.sol
contracts/src/migration
contracts/src/WLSETH.1.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Note: some contracts were not in context, but some bugs were still found (e.g. H-01 in WLSETH).

Protocol Overview

Liquid Collective is a liquid staking protocol that allows institutions to stake ETH and receive **LsETH**, a token representing legal and beneficial ownership of the staked ETH and the rewards it generates. The protocol manages validator operations, reward accounting, withdrawals, and redemptions through a set of smart contracts while enabling LsETH to remain liquid and usable across DeFi.

This differential audit focused on the following changes:

- Contracts were upgraded to 0.8.33
- Unnecessary unchecked { ++i; } were removed
- Loop increments have been moved to declaration for better readability
- Refactor of the validator allocation selection logic across the **OperatorsRegistryV1**, **RiverV1**, and **ConsensusLayerDepositManagerV1** . Moving from a count-based approach to explicit operator allocation model, where allocations specify how many validators each operator should provide.
- Validator exit mechanism has been updated from a count-based approach to explicit operator allocation model, where allocations specify how many validators each operator should be exited.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Assets

- **LsETH Token Supply** – Liquid staking token representing claim on staked ETH and rewards.
- **wLsETH Token Supply** – Wrapped LsETH representation used for integrations and accounting.
- **Protocol Treasury / Fees** – Protocol fee share accumulated through staking rewards.
- **Validator Deposits (32 ETH per validator)** – Capital allocated to validators via deposit contract. EIP-7251 will be implemented in a future upgrade.
- **Validator Key Registry** – Set of validator pubkeys registered in [OperatorsRegistry](#).
- **Validator Allocation Limits** – Operator-specific limits determining how many validators can be funded.
- **Pending Withdrawal ETH** – ETH exiting validators and waiting to satisfy redemption requests.
- **Redeem Queue State** – Accounting for pending LsETH redemption requests.
- **Committed Balance for Deposits** – ETH committed to future validator deposits.
- **Slashing Coverage Fund** – ETH reserve intended to absorb validator slashing losses.
- **Protocol Governance / Admin Control** – Authority to configure operators, limits, and upgrades.
- **Allowlist / Denylist State** – Address permission system controlling transfers and protocol access.
- **Oracle Reports and Consensus State** – Data reported by oracle regarding validator counts and balances.
- **Operator Validator Keys and Exit Requests** – Tracking of validator lifecycle states (funded, requested exit, stopped).

Actors

- **Institutional Stakers / Users**
 - Deposit ETH and receive LsETH.
 - Can wrap LsETH into wLsETH or redeem for ETH.
 - **Denied / Sanctioned Users**
 - Addresses placed on the denylist that should be prevented from interacting with the protocol.
 - **Protocol Administrator**
 - Manages operators, limits, allowlists, oracle members, and protocol parameters.
 - **Keeper**
 - Executes operational actions such as validator deposits or exit assignments.
 - **Oracle Committee**
 - Reports validator balances, counts, and other consensus-layer data.
 - **Validator Operators**
 - Entities responsible for running validators and managing validator keys.
 - **Off-chain Monitoring Infrastructure**
 - Indexers and monitoring bots relying on emitted events to track system state.
 - **Firewall Contract**
 - Access control layer routing administrative actions to protocol contracts.
-

Trust Assumptions

- **Oracle Committee Integrity**
 - Oracle members are assumed to report correct validator balances and counts.
 - The oracle is assumed not to manipulate validator accounting.
- **Validator Operator Honesty**
 - Operators are assumed to run validators correctly and process exit requests when assigned.
- **Validator Key Auditing**
 - Off-chain audit processes are assumed to verify validator pubkey uniqueness before funding.
- **Administrator Good Faith**
 - Admins are assumed not to abuse governance privileges or intentionally censor users.
- **Keeper Reliability**
 - Keepers are assumed to properly execute deposit and exit operations.
- **Event-based Monitoring Accuracy**
 - Off-chain monitoring systems are assumed to receive accurate state changes via emitted events.
- **Beacon Chain Correctness**
 - Ethereum deposit contract behavior and validator lifecycle rules are assumed to be correct.

Note: trust assumptions are guidelines, but if the audit finds a way to strengthen the system by limiting trust in external actors, we can submit those bugs.

Attack Vectors

- **Denylist Bypass via Wrapper Burn**
 - Denied users can unwrap wLsETH through `burn()` since allowlist checks were not applied to mint/burn operations.
 - Enables frozen accounts to recover LsETH and exit positions.
- **Accounting Manipulation via Direct Token Transfers**
 - Direct LsETH transfers to the wrapper inflate `totalSupply()` without updating internal accounting.
 - Creates stranded underlying tokens and breaks supply accounting.
- **Validator Key Duplication Attack**
 - Duplicate validator pubkeys can be registered due to missing uniqueness checks.
 - Deposit contract treats duplicate deposits as validator top-ups.
 - Causes phantom validator accounting and permanent asset inflation.
- **Phantom Asset Balance Inflation**
 - Incorrect validator count assumptions cause `_assetBalance()` to overestimate total assets.
 - Leads to:
 - Underpriced shares for new depositors.
 - Overpriced redemptions for existing holders.
 - Protocol fee distortions.
- **Denylist Bypass Through Redeem Flow**
 - River contract passes its own address as the initiator when redeeming.
 - Deny checks are applied to the contract rather than the actual user.
 - Allows denied users to claim redemption after being restricted.
- **Administrative Lockout via Firewall Selector Collision**
 - Firewall inherits `Administrable` and intercepts admin management calls.
 - Prevents changing admin of destination contracts.
 - Requires proxy upgrade to recover ownership.
- **Slashing Containment Bypass**
 - Slashing containment mode stops exit requests but still allows new validator deposits.
 - Capital can continue being exposed to compromised operators during incidents.
- **Exit Request Accounting Manipulation**

- Exit requests counted as “in-flight liquidity” even if operators fail to process them.
- Creates phantom liquidity that suppresses new exit demand.
- **Operator Exit Non-Compliance**
 - Operators failing to process exit requests accumulate unresolved exit demand.
 - System lacks mechanism to reclaim or reassign unfulfilled exits.
- **Operational Censorship Risk**
 - Admin or firewall misconfiguration could block governance actions or contract control.
- **Off-chain Monitoring Divergence**
 - Missing events cause monitoring systems to track incorrect operator limits.
- **Operator Allocation Logic Abuse**
 - Removal of validator attribution limits may allow disproportionate validator allocation per operator.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	1	1	1
Medium	2	2	1
Low	4	4	2
Informational	2	2	1
Total	9	9	4

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
Low	Low	Low	Low	Medium
	Low	Medium	High	
Likelihood				

Detailed Findings

ID	Title	Severity	Status
H-01	Denylisted users can bypass freeze controls by unwrapping wLsETH via burn	High	Fixed
M-01	Missing On-Chain Validator Pubkey Uniqueness Allows Permanent Asset Balance Inflation	Medium	Acknowledged
M-02	Inconsistent initiator deny check allows bypass via River redeem path	Medium	Fixed
L-01	Slashing containment mode still allows fresh validator funding	Low	Fixed
L-02	Phantom Pre-Exiting Balance From Unfulfilled Exit Requests	Low	Acknowledged
L-03	Firewall intercepts Administrable functions, locking destination contract ownership until proxy upgrade	Low	Acknowledged
L-04	Direct LsETH transfers to the wrapper strand underlying and break wLsETH supply accounting	Low	Fixed
I-01	`removeValidators` silently modifies operator limit without emitting SetOperatorLimit event, creating divergence between on-chain state and off-chain monitoring	Informational	Fixed

[I-02](#)

MAX_VALIDATOR_ATTRIBUTION_PER_ROUND
limitation has been removed, without any
documentation

Informational

Acknowledged

High Severity Issues

H-01: Denylisted users can bypass freeze controls by unwrapping wLsETH via burn

Severity: High	Impact: Medium	Likelihood: High
WLSETH.1.sol Rivier.1.sol	Status: Fixed	

Description:

The **WLSETH** contract checks allowlist status during transfers but not during **mint** or **burn** operations regarding the **msg.sender**. A denylisted user can call **burn**, which transfers underlying LsETH from the **WLSETH** contract to a recipient:

```
if (!river.transfer(_recipient, _shares)) {
```

The River contract sees the sender as **WLSETH** (not the denied user), allowing the transfer to proceed and enabling denied users to unwrap their tokens and exit their frozen positions:

```
function _onTransfer(address _from, address _to) internal view override {  
    IAllowlistV1 allowlist = IAllowlistV1(AllowlistAddress.get());  
  
    if (allowlist.isDenied(_from)) {  
        revert Denied(_from);  
    }  
}
```

Recommendations:

Enforce allowlist checks in **mint** and **burn** functions, specifically validating that **msg.sender** and the recipient are not denied.

Customer Response: Fixed in [PR-378](#).



Fix Review: Fix confirmed

Medium Severity Issues

M-01: Missing On-Chain Validator Pubkey Uniqueness Allows Permanent Asset Balance Inflation

Severity: Medium	Impact: High	Likelihood: Low
OperatorsRegistry.1.sol::addValidators() ConsensusLayerDepositManager.1.sol::depositToConsensusLayerWithDepositRoot() River.1.sol::_assetBalance()	Status: Acknowledged	

Description:

`addValidators()` stores validator public keys sequentially without checking uniqueness -- not within the submitted batch, not within the operator's existing keys, and not across operators. The only gate preventing a duplicate key from being funded is an unspecified off-chain audit performed before the admin calls `setOperatorLimits()`.

If a duplicate key passes the audit and gets funded, the Ethereum deposit contract does not reject it. It processes the deposit as a top-up to the existing validator's balance. However, `DepositedValidatorCount` increments for every deposit made, regardless of whether the key was already deposited.

`_assetBalance()` uses the difference between `DepositedValidatorCount` and the oracle-reported `clValidatorCount` to account for validators that have been deposited but not yet seen on the beacon chain:

```
validatorsBalance + (depositedValidatorCount - clValidatorCount) * 32 ETH
```

With one duplicate key funded, the beacon chain has N validators but `DepositedValidatorCount` equals N+1. The oracle can only ever report N for `clValidatorCount` (enforced by the constraint `_report.validatorsCount <= DepositedValidatorCount` in `OracleManager`). The system

permanently computes an extra 32 ETH of phantom balance that does not correspond to any real asset.

This phantom balance inflates the return value of `_assetBalance()`, which feeds into `_balanceFromShares()` and `_sharesFromBalance()`. The consequences are:

- New depositors receive fewer shares per ETH (overpaying for entry).
- Existing holders can redeem at an inflated rate, extracting more ETH than their shares are worth.
- Protocol fee calculations are distorted.

The inflation is irreversible. `DepositedValidatorCount` has no decrement function. `removeValidators()` reverts for any key index below `operator.funded`. No admin mechanism exists to correct the count.

Recommendations:

Add a global pubkey uniqueness mapping in `OperatorsRegistry`:

```
mapping(bytes32 => bool) internal _registeredKeys; // keccak256(pubkey) =>
registered
```

Cost is one SSTORE per key (~20K gas), negligible relative to the 32 ETH each key controls. This makes the uniqueness invariant enforceable on-chain rather than depending solely on an off-chain audit with no recovery path if it fails.

Customer Response:

Acknowledged

"The suggested fix only works for 0x01 validators (no top-ups), and doesn't work when we have top-ups. We are currently working on enabling 0x02 validators which would change the deposit flow. As a result we are happy with the current off-chain checks that detect the duplications."

M-02: Inconsistent initiator deny check allows bypass via River redeem path

Severity: Medium	Impact: Medium	Likelihood: Medium
River.1.sol RedeemManager.1.sol	Status: Fixed	

Description:

When redeeming via the River contract, the `initiator` recorded in `RedeemManager` is the River contract address, not the user. Since the River contract is never denied, the initiator deny check in `_claimRedeemRequests` is effectively bypassed. This allows an address that becomes denied after requesting a redeem to still successfully claim their redemption (provided the recipient address is not denied).

```
redeemRequests.push(  
  RedeemQueueV2.RedeemRequest({  
    height: height,  
    amount: _lsETHAmount,  
    recipient: _recipient,  
    initiator: msg.sender, // @audit here msg sender is the river address,  
not the user address.  
    maxRedeemableEth: maxRedeemableEth  
  })  
);
```

Recommendations:

Pass the actual user address as the initiator when calling `RedeemManager.requestRedeem` from River.

Customer Response:

Fixed in [PR-379](#)

Fix Review:

Fix confirmed

Low Severity Issues

L-01: Slashing containment mode still allows fresh validator funding

Severity: Low	Impact: Low	Likelihood: Low
OperatorsRegistry1.sol	Status: Fixed	

Description:

When the oracle sets `slashingContainmentMode`, River only short-circuits new validator exit requests. The report flow still reaches `_commitBalanceToDeposit()`, moving idle ETH into `CommittedBalance`. Subsequently, the trusted keeper can still call `depositToConsensusLayerWithDepositRoot()` because the deposit path never checks the containment flag. Additionally, operators with recently slashed validators are not automatically marked unfundable. This means the protocol's emergency slashing mode does not stop new capital from being deployed into potentially compromised operators during an active incident, materially amplifying exposure exactly when the protocol intends to contain losses.

Recommendations:

Make slashing containment a hard gate on the funding path: make `_commitBalanceToDeposit()` a no-op while containment is active; revert `depositToConsensusLayerWithDepositRoot()` while containment is active; and consider explicitly blocking operators with newly stopped/slashed validators from receiving fresh funds.

Customer Response:

Fixed in [13d3add919](#) and [fb8b66596](#).

Fix Review:

Fix confirmed

Note: Apart from the fix for this issue, the PR is additionally disabling the following functions during the slashing containment mode.

- `requestRedeem()`



- *deposit()*

L-02: Phantom Pre-Exiting Balance From Unfulfilled Exit Requests

Severity: Low	Impact: Medium	Likelihood: Low
OperatorsRegistry1.sol#requestValidatorExits River1.sol# requestExitsBasedOnRedeemDemandAfterRebalancings	Status: Acknowledged	

Description:

When the protocol needs validators to exit to satisfy redemption demand, River calls `demandValidatorExits`, which increments `CurrentValidatorExitsDemand`. The keeper then calls `requestValidatorExits` to assign those exits to specific operators. This atomically moves the count from `CurrentValidatorExitsDemand` into `TotalValidatorExitsRequested` and increments each operator's `requestedExits`.

River uses `getStoppedAndRequestedExitCounts()` to compute a `preExitingBalance` — ETH that has been requested for exit but not yet confirmed as stopped:

River1.sol:

```
(uint256 totalStoppedValidatorCount, uint256 totalRequestedExitsCount) =
    or.getStoppedAndRequestedExitCounts();

uint256 preExitingBalance =
    (totalRequestedExitsCount > totalStoppedValidatorCount
     ? (totalRequestedExitsCount - totalStoppedValidatorCount)
     : 0) * DEPOSIT_SIZE;
```

This `preExitingBalance` is treated as liquidity that is already on its way and suppresses the creation of new exit demand. If an operator fails to process the requested exits due to an operational issue (key access failure, monitoring gap, tooling bug, infrastructure outage), their assigned exits might be delayed (or in the worst case are never fulfilled), but the accounting permanently records them as in-flight. Since `TotalValidatorExitsRequested` is only ever incremented and there is no admin function to reverse an unfulfilled assignment, the phantom

balance persists until the operator eventually triggers the exits off-chain — something the protocol cannot enforce or guarantee.

Two secondary issues compound the problem:

1. `requestValidatorExits` does not check whether an operator has fulfilled its previous exit requests (`stopped < requestedExits`), unlike `_getFundedCountForOperatorIfFundable` which enforces this for deposits. The keeper can inadvertently assign more exits to an already-lagging operator, amplifying the phantom balance.
2. The `active` flag in `requestValidatorExits` blocks exit assignment to deactivated operators. If an operator is deactivated while it still has funded validators, those validators cannot be targeted for exits, stranding their stake.

Recommendations:

Add an admin-gated `reclaimUnfulfilledExitRequests` function that, for each specified operator, resets `requestedExits` to the actual `stopped` count and moves the difference back into `CurrentValidatorExitsDemand`. This gives the admin a recovery path to clear phantom demand and allows the keeper to reassign exits to a different operator.

Additionally, in `requestValidatorExits`, replace the `active` check with a compliance guard that mirrors the one already present on the deposit path. This prevents piling exits on non-compliant operators while still allowing exit requests to reach inactive operators that have live validators on the beacon chain:

```
// Before (blocks inactive operators, allows non-compliant ones):
OperatorsV2.Operator storage operator = OperatorsV2.get(operatorIndex);
if (!operator.active) {
    revert InactiveOperator(operatorIndex);
}
if (count > (operator.funded - operator.requestedExits)) {

// After (blocks non-compliant operators, allows inactive ones):
OperatorsV2.Operator storage operator = OperatorsV2.get(operatorIndex);
if (_getStoppedValidatorsCount(operatorIndex) < operator.requestedExits) {
    revert OperatorIgnoredExitRequests(operatorIndex);
}
if (count > (operator.funded - operator.requestedExits)) {
```

Customer Response:

Acknowledged

"This is by design. We only work with reputable NOs with well defined SLAs that ensures the timely exits. We are working on enabling Ox02 validators which changes the risk here and we are happy with our"

L-03: Firewall intercepts Administrable functions, locking destination contract ownership until proxy upgrade

Severity: Medium	Impact: Low	Likelihood: High
Firewall.sol	Status: Acknowledged	

Description:

The **Firewall** contract inherits **Administrable**. When an admin attempts to call **proposeAdmin** or **acceptAdmin** on a destination contract through the Firewall, the Firewall's own implementation intercepts the call. This makes it impossible to change the admin of the underlying destination contracts via standard calls once the Firewall is set as their admin. Recovering admin control of the destination contracts requires a proxy upgrade with migration logic.

Recommendations:

Remove **Administrable** inheritance from the Firewall or use distinct function selectors for managing the Firewall's own admin.

Customer Response:

Acknowledged

"This is a deliberate design choice for the setup we use. The contracts were designed with the assumption that the firewall would always be the admin."

L-04: Direct LsETH transfers to the wrapper strand underlying and break wLsETH supply accounting

Severity: Medium	Impact: Medium	Likelihood: Medium
WLSETH.1.sol	Status: Fixed	

Description:

`wLsETH.totalSupply()` is derived dynamically from the wrapper contract's raw LsETH balance in River, while individual ownership is tracked in an internal `BalanceOf` mapping. If a user directly transfers LsETH to the wrapper contract via `River.transfer()`, the wrapper's overall River balance increases without crediting any user internally. This inflates `wLsETH.totalSupply()` and permanently strands the directly transferred LsETH, as normal burn flows cannot access unrecorded balances.

Recommendations:

Track the wrapper's accounted underlying explicitly using a variable updated only during mint and burn operations, rather than deriving it from the raw River balance.

Customer Response:

Fixed in [PR-380](#).

Fix Review:

Fix confirmed

Informational Severity Issues

I-01: `removeValidators` silently modifies operator limit without emitting `SetOperatorLimit` event, creating divergence between on-chain state and off-chain monitoring

Files:

[OperatorsRegistry.l.sol](#)

Description:

The `removeValidators` function contains code paths that modify `operator.limit` without emitting a `SetOperatorLimit` event. Off-chain systems (indexers, dashboards, monitoring bots, and key-auditing pipelines) that track operator limits exclusively via events will carry a stale limit value after `removeValidators` reduces it. This divergence affects only off-chain observability and does not corrupt on-chain state integrity.

Recommendations:

Emit the `SetOperatorLimit` event in both limit-adjustment branches of `removeValidators`.

Customer Response:

Fixed in [PR-337](#)

Fix Review:

Fix confirmed

I-02: MAX_VALIDATOR_ATTRIBUTION_PER_ROUND limitation has been removed, without any documentation

Description:

The previous implementation used to enforce the following limitation to allocations per operator:

```
uint256 pickedKeyCount = LibUint256.min(  
    LibUint256.min(  
        operators[selectedOperatorIndex].limit  
        - (operators[selectedOperatorIndex].funded +  
operators[selectedOperatorIndex].picked),  
        MAX_VALIDATOR_ATTRIBUTION_PER_ROUND  
    ),  
    _count  
);
```

However, this limitation has been removed, without any documentation.

Recommendations:

Consider documenting the removal of the `MAX_VALIDATOR_ATTRIBUTION_PER_ROUND`.

Customer Response:

Acknowledged

"Since we no longer use the on-chain allocation algorithm, there is no need for the comment."

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.